



7

Jogo de Exploração RPG

Documento Técnico — Versão 1.0 | HTML5 + CSS3 + JavaScript Vanilla

CLIENT SIDE

STATE MACHINE

GRID 8×8

Objetivo e Estrutura Geral

Objetivo

RPG de exploração em grade (grid) desenvolvido inteiramente em HTML, CSS e JavaScript puro, sem frameworks externos. O jogador explora um mapa desconhecido, encontra criaturas e baús, coleta equipamentos, participa de combates por turno e evolui até concluir toda a exploração do mapa. O projeto utiliza uma arquitetura baseada em máquina de estados (State Machine), controlando todas as ações do jogador através de estados definidos.

Três Camadas Principais

HTML

Sidebar esquerda (inventário, status, informações do inimigo), área principal (título, console, tabuleiro 8×8, controles)

CSS

Identidade visual retrô, scanlines CRT, gradientes, texturas de grama

JavaScript

Estado do jogo, renderização, exploração, combate, inventário, eventos e máquina de estados

Interface Gráfica e Sistema Visual

Especificações da Interface

Resolução fixa de 850×850 pixels dividida em:

- Sidebar Esquerda: Inventário, HP, ATK, DEF e informações do inimigo durante combate
- Área Principal: Nome do jogo, console de mensagens, tabuleiro 8×8 e controles

O mapa utiliza notação semelhante ao xadrez (A1, B4, C7, H8). A posição inicial do jogador é A1 (internamente: row = 7, col = 0).

Identidade Visual Retrô

Inspirada em RPGs dos anos 90, com tema escuro e fonte pixelada. Marcadores por emoji:

🧑 Jogador, 🧛 Inimigo, 📦 Baú.



Tema Escuro

Fonte pixelada e atmosfera sombria



Scanlines CRT

Simulação de monitor antigo



Barra de HP

Gradientes e texturas de grama

Sistema de Mapa e Matrizes

O mapa possui 8 linhas $\times$ 8 colunas, totalizando 64 células. Cada célula pode ser: 0 = segura, 1 = criatura, 2 = baú. Durante a geração procedural: 70% células vazias, 15% criaturas e 15% baús. A posição inicial é sempre segura.

1

logicMatrix

Responsável pela lógica do jogo. Armazena o estado de cada célula: 0 (segura), 1 (criatura) ou 2 (baú).

2

visualMatrix

Responsável pela aparência visual. Gerencia os estados de cada célula: unexplored, explored e current.

3

entityMatrix

Armazena objetos permanentes como goblin, chest ou null. Desacopla a representação gráfica da lógica interna.

Jogador, Inventário e Raridade

Atributos do Jogador

O estado inicial do jogador contém: HP = 100, ATK = 2, DEF = 0. A estrutura também inclui HP máximo e o Amuleto da Vida (item raro passivo).

Inventário

Os itens ficam armazenados em um vetor

`inventory[]`. Cada item possui: nome, tipo, raridade, efeito e atributos.

Sistema de Raridade

○ Comum — 60%

Espada de Madeira, Armadura Velha, Escudo, Pinga

● Médio — 25%

Espada Velha, Espada Amaldiçoada, Armadura Amaldiçoada, Bandagens

□ Raro — 15%

Espada do Anjo, Armadura do Anjo, Escudo Angelical, Amuleto da Vida

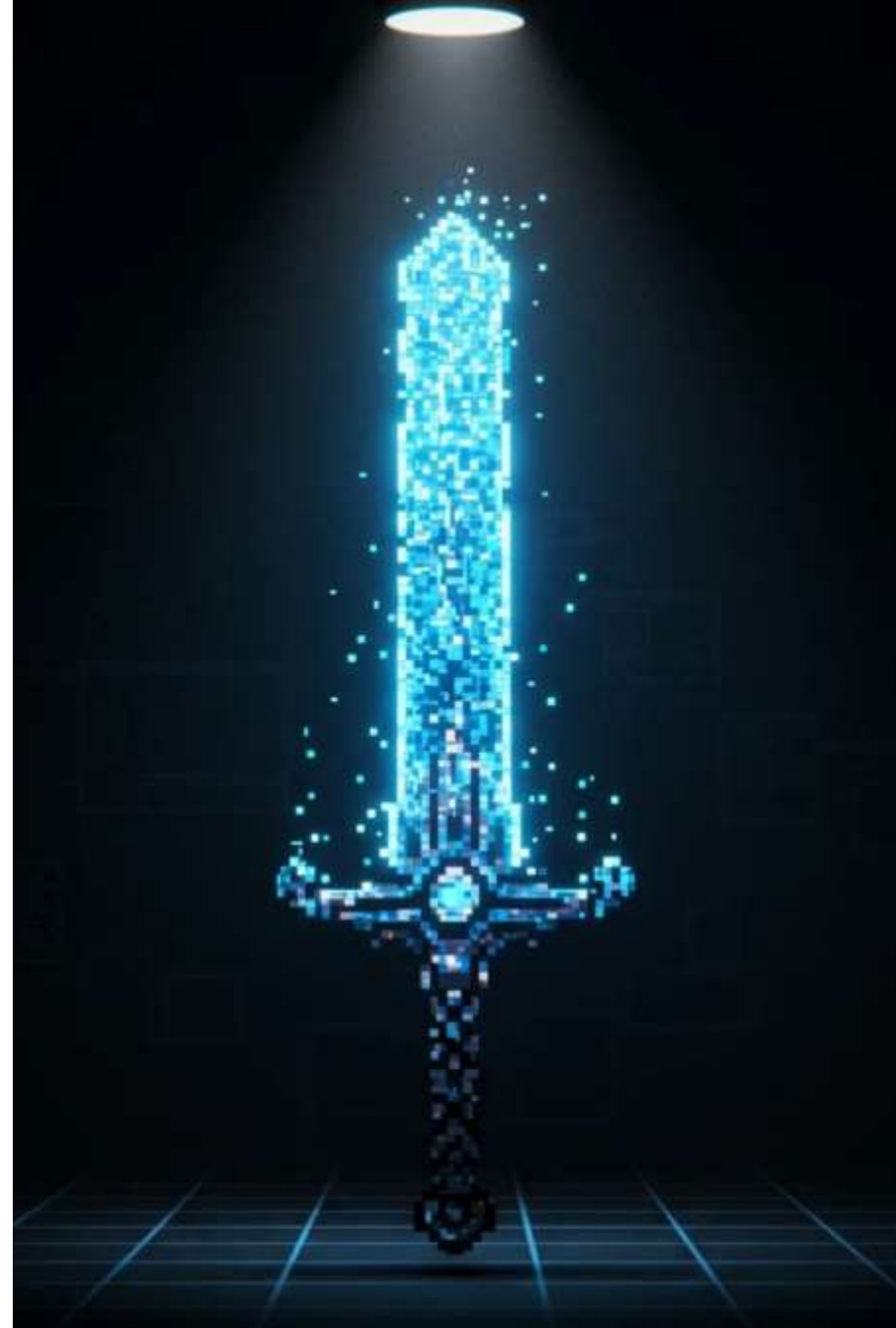
Exploração e Mimicry

Movimentação e Eventos

O jogador se movimenta com as setas ($\uparrow$ $\downarrow$ $\leftarrow$ $\rightarrow$). Caso a casa seja segura, o jogador avança e a área é revelada. Caso haja evento, o jogo solicita confirmação: Prosseguir? [Y] [N].

Mecânica de Mimicry

Existe uma mecânica de inversão aleatória. Ao entrar em uma célula perigosa: 50% continua igual e 50% troca: criatura $\leftrightarrow$ baú. Isso adiciona imprevisibilidade à exploração.



Sistema de Combate

Fluxo por Turnos

O combate é baseado em turnos: Jogador → Inimigo → Jogador → Inimigo. A cada turno, o jogador escolhe entre Atacar ou Defender.

- Ataque: Dano básico = ATK do jogador. Caso o inimigo esteja defendendo: $\text{dano} = \text{ATK} - \text{DEF inimigo}$
- Defesa: O jogador recebe redução de dano usando DEF. Após três defesas consecutivas: fuga automática

Sistema de Fuga

Antes do combate começar, a chance de fuga é calculada como: $\text{chance} = 1 - (\text{casas exploradas} / \text{total})$

Quanto menos explorado o mapa, maior a chance de fugir.

Inimigos, Baús e Consumíveis

Monstros Disponíveis

Os monstros são escolhidos aleatoriamente entre: Goblin, Bugbear, Owlbear, Displacer Beast, Kobold, Hobgoblin, Gnoll, Troll, Ogre, Minotaur, Basilisk e Manticore. Os atributos são gerados próximos aos atributos atuais do jogador.

Baús e Consumíveis

Ao abrir um baú, é gerado um item aleatório, adicionado ao inventário, com efeitos permanentes ou consumíveis. Equipamentos são aplicados imediatamente ao jogador, podendo alterar ATK, DEF e HP máximo. Alguns possuem maldição.



Pinga

HP +10, DEF -2



Bandagem

HP +15



Amuleto da Vida

Item raro passivo. Após vencer combate: +20 HP até o HP máximo.

Máquina de Estados e Controles

O jogo inteiro é controlado pela variável

`state`. Cada estado aceita apenas determinadas entradas do teclado.

Estados Disponíveis

- `explore` – Exploração do mapa
- `confirm_move` – Confirmação de movimento
- `combat_init` / `combat_turn` – Combate
- `use_item` – Uso de itens
- `victory` / `gameover` – Fim de jogo
- `new_game_plus` – Nova floresta

Mapeamento de Controles

Setas

Movimento

Y / N

Confirmar ou Cancelar

Espaço

Abrir inventário

↑↓ + Enter

Selecionar e usar item

Vitória, New Game Plus e Tecnologias

Condição de Vitória

Quando todas as 64 células forem exploradas, o jogador vence. O jogo oferece a opção de continuar para uma nova floresta.

New Game Plus

Ao aceitar: novo mapa gerado, exploração reiniciada, geração incrementada. O jogador mantém atributos e inventário existentes.

Tecnologias Utilizadas

- HTML5, CSS3, JavaScript ES6 CSS
- Variables, Grid e Flexbox DOM API e
- Eventos de Teclado Programação
- Procedural
- Máquina de Estados
- Geração Pseudoaleatória (`Math.random()`)

Considerações Técnicas

Arquitetura simples, porém bem organizada, com separação lógica entre renderização, regras de negócio, combate, exploração e inventário. A utilização de múltiplas matrizes desacopla a representação gráfica da lógica interna, facilitando futuras expansões como novos inimigos, NPCs, missões ou mapas maiores. A máquina de estados centraliza o fluxo de execução, reduz conflitos entre entradas do teclado e torna o comportamento do jogo previsível e extensível.